

从 Workload 到数据结构选择

把“会背结构名”升级为可审计、可反证的组合设计能力

408 · 数据结构 Integration I01

完整组合问题：先写契约，再拼能力

Workload → API Contract → Required Capabilities → Candidate Structures
→ Composite State / Invariants → Cost Vector → Choice → Boundary Check.

本册不重新教授任何单一结构，而是检查能否把 Atlas Foundation、Topic 和 Bridge 的接口组合成一项有理由、可反证、可迁移的设计。一个 API 往往要求多种能力；单一结构不能同时提供时，正确动作不是寻找“万能结构”，而是让多个 Owner 各自承担一种能力，并显式维护它们之间的同步不变量。

Position、Owns、Uses 与 Stop Boundary

- **Position:** 本册是数据结构 Integration，拥有“从需求到组合方案”的完整决策轨迹。
- **Owns:** API/工作负载解析、能力分解、结构组合、跨结构同步不变量、规模升级路线、条件化选择与独立验证。
- **Uses:** DS01 的数组/链表表示，DS05 的优先队列，DS09 的有序索引，DS10 的 Hash，DS12 的外部排序，以及 DS-B02 的 Workload-Index 比较接口。
- **Stops:** Hash 冲突、堆上滤/下滤、链表改边、外部归并等局部机制回各 Topic；线程调度、JVM GC、MapReduce 执行引擎只作为工程 Extension，不在本册重定义。

若缺少操作频率、最坏保证、数据分布或资源约束，应输出条件化选择，而不是伪造唯一答案。

目录

1	先把自然语言需求改写成可检查的合同	3
1.1	API 名字不等于真正需求	3
1.2	能力分解：每个成本承诺需要谁来兑现	3
1.3	不要先追求最优：先写可证明的基线	3
2	组合结构的核心不是容器数量，而是同步不变量	4
2.1	一份事实可以有多个索引，但不能有多个真相	4
2.2	原子状态转移清单	4
3	生成性例一：Hash + Array 实现随机集合	4
3.1	为什么“交换到末尾再删除”是必要接口	4
4	生成性例二：Hash + 双向链表实现 LRU	5
4.1	能力分工	5

4.2	状态与不变量	5
4.3	为什么使用哨兵	5
5	生成性例三：LFU 的二维顺序	6
6	动态排名：Top-K、Quickselect 与双堆中位数	6
6.1	Top-K 不是“看到 K 就用堆”	6
6.2	局部 Top-K 可以合成全局 Top-K	6
6.3	频率 Top-K：先聚合，再维护淘汰门槛	6
6.4	动态中位数：分区不变量比两个堆的名字重要	7
6.5	关注流信息流：Hash 定位 + 时间线 + K 路归并	7
7	规模升级：从内存结构到外存与分布式处理	7
7.1	第一问不是“用什么算法”，而是“状态放得下吗”	7
7.2	四级规模模型	7
7.3	Hash 分片的正确性条件	8
7.4	存在性、去重、交集与频率是不同合同	8
8	外部排序与范围分桶：瓶颈必须测量	8
8.1	外部归并的 Owner 组合	8
8.2	桶排序的成立条件	8
8.3	CPU、I/O、内存和串行尾部没有固定赢家	9
9	完整决策表与贯穿案例	9
9.1	七步决策表	9
9.2	贯穿例子一：日志索引	9
9.3	贯穿例子二：最近十条动态	9
10	反证、失败边界与出口	10
11	验证协议：不仅验证结果，也验证状态	10
12	做题控制协议与压缩复原	10

1 先把自然语言需求改写成可检查的合同

1.1 API 名字不等于真正需求

“设计缓存”“维护热搜”“做一个随机集合”仍然太粗。开始选结构前，应把每个操作写成五元组：

(Input, Output, Mutation, Frequency, Required Bound).

例如随机集合的合同不是“支持集合操作”，而是：

操作	语义	状态变化	目标成本
insert (x)	若不存在则加入并返回真	增加一个可随机抽样元素	平均 $O(1)$
remove (x)	若存在则删除并返回真	删除后其余元素仍可随机抽样	平均 $O(1)$
getRandom ()	对现存元素做等概率抽样	只读	$O(1)$

“等概率”会生成数组的紧凑下标能力，“存在性与定位”会生成 Hash 的直接定位能力，“任意位置删除”则产生二者之间的交接压力。

1.2 能力分解：每个成本承诺需要谁来兑现

常见能力与默认候选如下：

所需能力	常见 Owner	能得到什么	必须付出什么
按 key 直接定位	Hash	平均常数查找	冲突、装填、重散列、无序
按位置随机访问	紧凑数组	$O(1)$ 下标访问	中间插删需移动
已知节点局部删除	双向链表	$O(1)$ 改边	需要额外定位与指针空间
持续取得当前极值	Heap / Priority Queue	查看极值 $O(1)$ 、更新 $O(\log n)$	不提供任意排名和全序
范围、前驱、后继	有序索引	有序查询与扫描	更新需维护有序/平衡
前缀匹配	Trie	查询成本绑定 key 长度	节点稀疏与空间放大
超内存排序/聚合	分片、外部归并	把内存约束转成多阶段处理	I/O、缓冲、临时文件与倾斜

这张表不是答案表。它只给候选；最终选择仍由操作比例、最坏保证、输入分布、内存、I/O 与是否允许近似共同决定。

1.3 不要先追求最优：先写可证明的基线

组合设计遵循三步：

1. 写出语义正确的基线，例如全量排序、线性扫描或单一 Map；
2. 指出基线重复了什么工作或违反了哪一项资源约束；
3. 只增加能消除该瓶颈的状态，并写出新增状态的维护成本。

“换成更高级的数据结构”不是推导。必须说明它预存了什么信息，为什么该信息足以减少重复工作，以及更新时怎样保持真实。

2 组合结构的核心不是容器数量，而是同步不变量

2.1 一份事实可以有多个索引，但不能有多个真相

组合结构通常保存一份逻辑对象和多个访问路径：

$$\text{Logical Record} \leftrightarrow \begin{cases} \text{Key Index,} \\ \text{Position / Order Index,} \\ \text{Priority / Frequency Index.} \end{cases}$$

真正的难点是所有索引必须指向同一批有效对象。常见同步不变量包括：

1. Map 中每个 key 恰好指向一个仍存活的节点或下标；
2. 数组有效区间 $[0, n)$ 没有空洞，Map 中记录的下标与数组一致；
3. 链表中每个真实节点恰好出现一次，头尾哨兵互相可达；
4. 频率桶内的 key 与 key 表中的频率一致，空桶不参与最小频率判断；
5. 两个堆的分区有序且大小差不超过一。

最危险的局部更新

组合结构中的 bug 往往不是某个容器操作错误，而是只更新了一个索引。例如数组尾元素搬到被删位置后若忘记更新 Map 中的下标，第一次删除看似成功，下一次定位就会访问错误对象。每个修改操作都应列出“哪些视图变了”，并在返回前逐一恢复不变量。

2.2 原子状态转移清单

即使实现语言没有事务，也可以按事务思维审查：

Validate Preconditions → Identify All Affected Views → Mutate
→ Repair Cross-Index Links → Check Size / Boundary.

若中途可能失败，还要决定回滚、延迟提交还是先构造新状态再交换。408 伪代码通常不要求异常安全，但顺序仍决定边界是否正确。

3 生成性例一：Hash + Array 实现随机集合

3.1 为什么“交换到末尾再删除”是必要接口

数组支持 $O(1)$ 随机下标，但任意中间删除会移动后缀。Map 能立即找到目标下标，却不能消除移动。解决方法是把“删除任意位置”翻译成“删除末尾”：

locate $i \rightarrow A[i] \leftarrow A[n-1] \rightarrow$ repair moved element's index $\rightarrow$ pop tail $\rightarrow$ erase key.

它牺牲元素顺序，换来紧凑数组与常数删除；若题目要求保持顺序，这个方案立即失效。

```
1 state:
2   values[0..n)           // dense, no holes
3   pos[value] -> index    // exact inverse index
4
5 insert(x):
```

```

6   if x in pos: return false
7   pos[x] = n
8   values.push_back(x)
9   return true
10
11 remove(x):
12   if x not in pos: return false
13   i = pos[x]
14   y = values[n - 1]
15   values[i] = y
16   pos[y] = i           // repair before/with shrinking
17   values.pop_back()
18   erase pos[x]
19   return true

```

代码清单 1: 随机集合的状态合同伪代码

当 x 本来就是末元素时, $y = x$; 先写回 $\text{pos}[y]$ 再删除 $\text{pos}[x]$ 仍可得到正确结果。空集合上的随机抽样必须由 API 预条件或显式失败语义处理。

4 生成性例二: Hash + 双向链表实现 LRU

4.1 能力分工

LRU 要求 get/put 都为平均 $O(1)$:

- Hash: 由 key 直接找到节点;
- 双向链表: 已知节点时 $O(1)$ 删除, 并把它移到“最近使用”一端;
- 容量计数: 超过 capacity 时从“最久未使用”一端淘汰。

单链表不能在仅持有当前节点时 $O(1)$ 找到前驱; 节点必须保存 key, 否则从链表淘汰尾节点后无法同步删除 Hash 项。

4.2 状态与不变量

设头哨兵之后是最近使用端, 尾哨兵之前是最久未使用端:

$\text{keys}(\text{Map}) = \text{keys}(\text{List}),$
 $\text{Map}[k]$ 指向链表中唯一的 k 节点,
 $\text{List order} = \text{nonincreasing recency},$
 $|\text{Map}| \leq \text{capacity}.$

get(k) 命中不仅返回值, 还会改变 recency, 因此是逻辑写操作; put 更新旧 key 时也必须刷新其最近使用位置。容量为零是独立边界, 任何插入都不能留下节点。

4.3 为什么使用哨兵

虚拟头尾把“空表、首节点、尾节点、中间节点”统一为同一种四指针改边, 减少分支但不消灭边界。仍需保证: 删除的是真实节点; 哨兵不进入 Map; 清空后 $\text{head.next} = \text{tail}$ 且 $\text{tail.prev} = \text{head}$ 。

5 生成性例三：LFU 的二维顺序

LFU 的淘汰序不是单轴，而是字典序：先比较 frequency，再在同频率内按 recency 淘汰最旧者。

$$\text{Eviction Priority} = (\text{frequency ascending, recency ascending}).$$

因此需要：

- keyMap: key → value, frequency 与节点位置；
- freqMap: frequency → 该频率下按 recency 排列的集合/链表；
- minFreq: 当前非空最小频率。

一次命中要把 key 从频率 f 的桶删除、必要时删除空桶、更新 minFreq，再把它插入 $f + 1$ 桶的最近端。插入新 key 后 minFreq=1。如果同频率容器只是无序集合，就无法兑现“频率相同淘汰最久未使用”的合同。

“所有操作 $O(1)$ ” 的隐藏前提

必须选用支持已知节点常数删除、并保持桶内顺序的结构。若每次为了找到同频率最旧项而扫描，淘汰不再是 $O(1)$ ；若用平衡树维护频率，界会变成 $O(\log n)$ 。复杂度声明必须与具体表示一致。

6 动态排名：Top-K、Quickselect 与双堆中位数

6.1 Top-K 不是“看到 K 就用堆”

选择取决于数据是否流式、是否允许改写、是否只查询一次：

场景	候选	成本	边界
流式维护最大 K 个	大小为 K 的最小堆	$O(N \log K)$ ，空间 $O(K)$	可随到随更
静态数组单字第 K 大	Quickselect	平均 $O(N)$ ，最坏 $O(N^2)$	会改写数组；随机 pivot 降风险
需要完整有序输出	排序	$O(N \log N)$	多做了全序但满足输出
值域小且已知	计数/桶	$O(N + R)$	成本绑定值域 R

“最大 K 个”用最小堆，是因为堆顶承担当前候选集合的淘汰门槛；新元素不超过门槛即可丢弃，超过门槛则替换。若求最小 K 个，方向反转为最大堆。

6.2 局部 Top-K 可以合成全局 Top-K

若数据被分成互不重叠的若干分片，每片只保留自己的 Top-K，全局 Top-K 一定包含在这些局部候选的并集中：任何未进入某片 Top-K 的元素，在同片至少有 K 个元素不小于它，因此不可能进入全局 Top-K。这给出分布式/分片合并的正确性，而不是只凭“MapReduce 常这样做”。

6.3 频率 Top-K：先聚合，再维护淘汰门槛

“出现次数最多的 K 个”不是直接对原始记录做大小堆：第一阶段必须用 Hash/分片 Map 将相同 key 聚合成 (key, count)，第二阶段才在频次流上维护大小为 K 的最小堆。堆顶是当前候选的最低频次；新计数超过门槛才替换。若频次相同，必须在堆比较器中固定 key 的字典序或首次出现时间，否则“第

K 个”没有确定合同。全量内存方案为 $O(N)$ 计数加 $O(U \log K)$ 堆维护 (U 为不同 key 数)；分片方案先在每片聚合并保留局部 Top- K ，再对局部候选重新聚合/合并，正确性依赖同 key 必须进入同片或在第二阶段再次合并。

Count-Min Sketch 可以用较小空间给出频次上界并筛出 heavy-hitter 候选，但哈希碰撞会造成高估；它不能单独承担精确 Top- K 的最终裁决。可行的工程合同是“Sketch 预筛 + 精确 Map 二次确认”，并把误差界、概率和 ties 写进 API。

6.4 动态中位数：分区不变量比两个堆的名字重要

用最大堆保存较小一半、最小堆保存较大一半，维护：

$$\max(L) \leq \min(R), \quad |L| - |R| \in \{0, 1\}.$$

插入时可先进入左堆，再把左堆最大值转移到右堆，若右堆更大再转回一个；这种固定修复顺序同时恢复分区有序和规模平衡。奇数个元素时中位数为 $\max(L)$ ，偶数时为两堆顶平均值；若数值可能溢出，求和前必须提升类型。

6.5 关注流信息流：Hash 定位 + 时间线 + K 路归并

“返回自己和所有关注者的最近十条动态”是一个典型组合接口。Hash 保存 $\text{user} \rightarrow \text{follows}$ 和 $\text{author} \rightarrow \text{timeline}$ ，每个作者的时间线保持按时间倒序；查询时把自己与关注者的时间线头元素放入最大堆，每次弹出最新一条，再只推进同一来源的下一条。若来源数为 F 、输出条数为 K ，查询成本约为 $O(F + K \log F)$ ，而不是把所有历史动态收集后全量排序；空间为堆中的 $O(F)$ 个 frontier。实现必须明确：用户自身是否总被包含、同一时间戳的 tie-break、follow/unfollow 后对后续查询的生效时刻、已删除动态是否跳过，以及分页游标如何避免重复返回。

全量收集再排序仍可作为小规模、低频查询的正确基线；当历史长度远大于 K 时， K 路归并才兑现了“成本绑定来源数与输出量”的承诺。这里的图关系、Hash 定位、时间序列和堆 frontier 各自有 Owner，I01 只拥有它们如何组合及跨视图一致性。

7 规模升级：从内存结构到外存与分布式处理

7.1 第一问不是“用什么算法”，而是“状态放得下吗”

对 N 条记录，先估算：

$$\text{Working Set} \approx N \times (\text{payload} + \text{index overhead} + \text{allocator/runtime overhead}).$$

不能只用源文件字节数判断 HashSet/HashMap 是否能装入内存；指针、桶、对象头、装填率和字符串对象可能把内存放大数倍。安全分片数应留出输入缓冲、输出缓冲、运行时与倾斜余量：

$$K \geq \left\lceil \frac{\text{Estimated Working Set}}{\text{Usable Memory per Worker}} \right\rceil.$$

7.2 四级规模模型

1. **Fits in memory**: 直接使用 Hash、Heap、排序或有序索引；
2. **Streaming**: 输入顺序扫描，只保存固定或次线性摘要，如 Top- K 小堆；
3. **Partitioned external**: 按 key/range 分片，使每片可装入内存后逐片处理；

4. **Distributed**: 各节点处理局部状态，随后按可证明的合并接口归约。

规模升级不是自动优化。每升一级都会增加临时文件、序列化、网络、失败恢复与倾斜处理成本。

7.3 Hash 分片 的正确性条件

去重、频率统计与交集问题常依赖：

$$x = y \implies h(x) \bmod K = h(y) \bmod K.$$

因此相同 key 必须进入同一编号分片。两个文件求交集时必须使用相同规范化规则、Hash 函数、种子与分片数，再逐对处理对应分片。Hash 冲突只让不同 key 落入同片，不会破坏完整性；最终相等判断仍必须比较真实 key，不能把 Hash 值相等当成对象相等。

数据倾斜可能让某片仍超内存，正确恢复是对该片使用新种子/更多位再次分片，或切换范围分区；“平均每片大小足够”不是最坏保证。

7.4 存在性、去重、交集与频率是不同合同

任务	精确方案主干	可选压缩	关键边界
存在性查询	HashSet / bitmap	Bloom filter	Bloom 有假阳性，不能单独证明存在
去重	Hash 分片后逐片 Set	排序后相邻去重	输出顺序、重复定义、规范化
两文件交集	同分片后 Set probe	外部排序 + 双指针	两侧必须同分片规则
频率统计	分片后 Map 计数	Count-Min Sketch	近似误差与二次确认
Top-K	固定小堆	近似 heavy hitter	ties、顺序与精确性

位图要求 key 能映射到可接受大小的稠密整数域；Bloom filter 用多个位位置换空间，允许“可能存在/一定不存在”，不允许把“可能存在”直接当最终精确答案。

8 外部排序与范围分桶：瓶颈必须测量

8.1 外部归并的 Owner 组合

当数据不能装入内存且要求全序，主线为：

$$\text{Run Generation} \rightarrow \text{Internal Sort per Run} \rightarrow \text{Buffered K-way Merge} \rightarrow \text{Ordered Output}.$$

多路归并用最小堆保存各 run 当前头元素，总代价约为 $O(N \log K)$ 次候选比较，并伴随顺序块 I/O。内存预算必须同时容纳 K 个输入缓冲、输出缓冲、堆节点和解析状态；不能把全部内存都分给 run 数据。

8.2 桶排序的成立条件

范围分桶只有在 key 域可分、桶序能推出全局序、且每桶可被继续处理时才成立。数据分布不均会制造巨大热桶；此时需要采样分位点、递归细分或回退外部归并。按桶编号拼接只在“任意前桶 key 不大于任意后桶 key”时成立，Hash 分片没有这个顺序性质，不能直接拼接成排序结果。

8.3 CPU、I/O、内存和串行尾部没有固定赢家

同样是外部处理，不同 workload 的瓶颈可能相反：

- 每条记录只解析整数并维护 $K = 100$ 的堆时，磁盘吞吐可能占主导；
- 每条记录要解析 JSON、提取字段并做昂贵比较时，CPU 可能占主导；
- 多线程同时加载大批对象时，内存与 GC 压力可能占主导；
- 局部排序已经并行，最终单路/少路归并仍可能形成串行尾部。

因此“外部排序一定 I/O bound”与“多线程一定更快”都是错误的无条件规则。先测每阶段的读取、解析、比较、写出、等待、峰值内存与并行度，再决定扩大 batch、增加线程、减少对象化还是改变算法。

实验数据不是通用定律

某次 10GB JSON 实验中的 CPU 利用率、耗时、JVM 参数和 GC 行为只能说明该硬件、格式、实现与参数下的瓶颈。Canonical 模型吸收“分阶段测量、定位主导成本和串行比例”的方法，不把具体秒数或百分比晋升为普遍事实。

9 完整决策表与贯穿案例

9.1 七步决策表

步骤	必须写出	验证问题
Workload	操作种类、频率、规模、分布、资源	是否遗漏主操作或阶段？
API Contract	输入、输出、修改、失败语义、成本界	“平均/最坏/均摊/近似”是否说清？
Capabilities	定位、顺序、极值、随机、前缀、外存	每项能力真的由某 Owner 兑现吗？
Candidates	至少一个正确基线和一个改进候选	每个候选满足语义合同吗？
Invariants	单结构与跨结构同步条件	每次更新后怎样恢复？
Cost Vector	时间、空间、比较、移动、I/O、通信	成本绑定哪个输入和资源层？
Choice / Boundary	取舍、假设、失败与升级路径	改变 workload 后选择会否反转？

9.2 贯穿例子一：日志索引

任务 A：百万条记录，绝大多数操作是精确 user_id 查找，偶尔插入，空间可扩展。候选为 Hash 与平衡树；Hash 以装填和冲突管理换取平均精确查询，平衡树提供最坏对数级和有序遍历。任务 B 增加时间范围扫描后，Hash 需额外维护排序结构，平衡树或 B+ 结构的选择理由增强。结构机制分别由 DS09/DS10 Own，比较接口由 DS-B02 Own。

9.3 贯穿例子二：最近十条动态

若每个用户维护按时间递增的动态序列，查询某用户及其关注者的最近十条，直接收集全部动态再排序是正确基线，但成本随历史总量增长。改进方案让每个来源只暴露自己的尾部迭代器，并用大小等于来源数的最大堆做 K 路合并；每弹出一条，只把同一来源的前一条加入候选。于是成本绑定关注来源数与输出条数，而不是全部历史。

这里 Hash 只负责 user -> follows/timeline 定位，时间序列负责来源内有序，Heap 负责跨来源 frontier。若关注关系、删除语义或分页游标改变，必须重新审查状态与一致性；不能把演示代码中“收集后全排序”的基线冒充最终伸缩方案。

10 反证、失败边界与出口

危险结论	最小反例/追问	正确出口
Hash 总是最快	范围查询、碰撞最坏界、重散列、顺序输出	回 DS09/DS-B02 比较
平衡树总是最好	只需精确查找且更新频繁，是否值得维护全序	允许条件化选择
看到 Top-K 就用堆	静态数组单次查询且允许改写	比较 Quickselect/排序
Hash 分片后可直接排序拼接	Hash 分片无范围顺序	使用范围桶或外部归并
多线程一定加速	文件读取串行、锁竞争、GC、最终归并	先做阶段测量
Bloom 说存在就一定存在	假阳性	二次精确确认
组合结构各自正确即可	两个索引可互相失配	写跨结构同步不变量

11 验证协议：不仅验证结果，也验证状态

组合结构至少使用以下测试：

1. 空结构、容量为零、单元素、重复插入、删除不存在 key；

2. 删除数组末元素与非末元素，随后再次定位被搬移元素；

3. LRU 更新旧 key、连续命中、容量淘汰与重新插入；

4. LFU 同频率 tie-break、桶清空、minFreq 跳转与容量为零；

5. Top-K 的 $K = 1$ 、 $K = N$ 、重复值与 ties；

6. 双堆中位数的递增、递减、重复、奇偶长度与数值溢出；

7. 分片后的全量守恒：输入记录数等于各分片记录数之和；

8. 相同 key 必须同片，逐片结果合并后与小规模内存基准一致；

9. 人工制造倾斜片、短读写、空 run 与最后不足一个 batch 的尾块。

可在小规模输入上同时运行“慢但显然正确”的基线与优化方案，比较输出集合/序列；这是验证组合正确性的独立信息，不应只用优化代码自证。

12 做题控制协议与压缩复原

草稿固定写九行：

1. API 与失败语义；

2. 操作频率与规模；
3. 每项操作要求的能力和复杂度口径；
4. 正确基线；
5. 基线重复工作/资源瓶颈；
6. 候选结构与能力分工；
7. 跨结构同步不变量；
8. 成本向量、假设与边界；
9. 最小反例和规模升级路线。

最后删除没有改变选择的装饰细节；保留能让下一道陌生题复用的接口句。

压缩复原：九问

真正昂贵的操作是什么？成本要求是平均、最坏还是均摊？哪个结构提前保存了所需信息？单一结构缺哪项能力？多个结构怎样指向同一份逻辑事实？一次修改要同步哪些视图？数据是否装得下？若装不下，按什么不变量分片并怎样合并？改变 workload 后选择是否反转？